NAME: MOAZZAM FAROOQUI
ROLL NO: CT-24068
COURSE CODE: CT-261
INSTRUCTOR: SIR ROHAIL QAMAR
TASK: DBMS LAB#07

Q1. Run the statement to build the MY_EMPLOYEE table to be used for the lab.

SQL QUERY:

```sql
CREATE TABLE MY_EMPLOYEE (
    id          NUMBER(4),
    last_name VARCHAR2(25),
    first_name VARCHAR2(25),
    userid     VARCHAR2(8),
    salary     NUMBER(9,2)
);

```

OUTPUT:

```
Table created.
```

Q2. Describe the structure of the MY_EMPLOYEE table to identify the column names.

SQL QUERY:

```
--2
DESC MY_EMPLOYEE;
```

OUTPUT:

Object Type TABLE Object MY_EMPLOYEE

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
MY_EMPLOYEE	ID	NUMBER	-	4	0	-	✓	-	-
	LAST_NAME	VARCHAR2	25	-	-	-	✓	-	-
	FIRST_NAME	VARCHAR2	25	-	-	-	✓	-	-
	USERID	VARCHAR2	8	-	-	-	✓	-	-
	SALARY	NUMBER	-	9	2	-	✓	-	-

Q3. Add the first row of data to the MY_EMPLOYEE table from the following sample data.
Do not list the columns in the INSERT clause.
ID LAST_NAME FIRST_NAME USERID SALARY
1 Patel Ralph rpatel 895
2 Dancs Betty bdancs 860
3 Biri Ben bbiri 1100
4 Newman Chad cnewman 750
5 Ropeburn Audrey aropebur 1550

SQL QUERY:

```
--3
INSERT INTO MY_EMPLOYEE
VALUES (1, 'Patel', 'Ralph', 'rpatel', 895);
```

OUTPUT:

```
1 row(s) inserted.
```

Q4. Populate the MY_EMPLOYEE table with the second row of sample data from the preceding list. This time, list the columns explicitly in the INSERT clause.

SQL QUERY:

```
--4
INSERT INTO MY_EMPLOYEE (id, last_name, first_name, userid, salary)
VALUES (2, 'Dancs', 'Betty', 'bdancs', 860);
```

OUTPUT:

```
1 row(s) inserted.
```

Q5. Confirm your addition to the table.

SQL QUERY:

```
--5
SELECT * FROM MY_EMPLOYEE;
```

OUTPUT:

ID	LAST_NAME	FIRST_NAME	USERID	SALARY
2	Dancs	Betty	bdancs	860
1	Patel	Ralph	rpatel	895

Q6. Write an INSERT statement to load rows into the MY_EMPLOYEE table. Concatenate the first letter of the first name and the first seven characters of the last name to produce the user ID.

SQL QUERY:

```
INSERT INTO MY_EMPLOYEE (ID, LAST_NAME, FIRST_NAME, USERID, SALARY)
VALUES (&ID, '&LAST_NAME', '&FIRST_NAME',LOWER(SUBSTR('&FIRST_NAME',1,1) || SUBSTR('&LAST_NAME',1,7)), &SALARY);
```

Q7. Populate the table with the next two rows of sample data by running the INSERT statement in the script that you created.

OUTPUT:

```
--7
INSERT INTO MY_EMPLOYEE (id, last_name, first_name, userid, salary)
VALUES (3, 'Biri', 'Ben', LOWER(SUBSTR('Ben', 1, 1) || SUBSTR('Biri', 1, 7)), 1100);


INSERT INTO MY_EMPLOYEE (id, last_name, first_name, userid, salary)
VALUES (4, 'Newman', 'Chad', LOWER(SUBSTR('Chad', 1, 1) || SUBSTR('Newman', 1, 7)), 750);
```

OUTPUT:

```
1 row(s) inserted.
```

```
1 row(s) inserted.
```

Q8. Confirm your additions to the table.

SQL QUERY:

```
--8
SELECT * FROM MY_EMPLOYEE
```

OUTPUT:

ID	LAST_NAME	FIRST_NAME	USERID	SALARY
2	Dancs	Betty	bdancs	860
4	Newman	Chad	cnewman	750
3	Biri	Ben	bbiri	1100
1	Patel	Ralph	rpatel	895

4 rows returned in 0.00 seconds Download

Q9. Make the data additions permanent.
(Cannot be implemented in Oracle Apex)

SQL QUERY:

```
--9
COMMIT;
```

Q10. Change the last name of employee 3 to Drexler.

SQL QUERY:

```
--10
UPDATE MY_EMPLOYEE
SET last_name = 'Drexler'
WHERE id = 3;
```

OUTPUT:

```
1 row(s) updated.
```

Q11. Change the salary to 1000 for all employees with a salary less than 900.

SQL QUERY:

```
--11
UPDATE MY_EMPLOYEE
SET salary = 1000
WHERE salary < 900;
```

OUTPUT:

```
3 row(s) updated.
```

Q12. Verify your changes to the table.

SQL QUERY:

```
SELECT * FROM MY_EMPLOYEE;
```

OUTPUT:

ID	LAST_NAME	FIRST_NAME	USERID	SALARY
2	Dancs	Betty	bdancs	1000
4	Newman	Chad	cnewman	1000
3	Drexler	Ben	bbiri	1100
1	Patel	Ralph	rpatel	1000

Q13. Delete Betty Dancs from the MY_EMPLOYEE table.

SQL QUERY:

```
DELETE FROM MY_EMPLOYEE
WHERE FIRST_NAME = 'Betty'
AND LAST_NAME = 'Dancs';
```

OUTPUT:

```
1 row(s) deleted.
```

Q14. Confirm your changes to the table.

SQL QUERY:

```
SELECT * FROM MY_EMPLOYEE;
```

OUTPUT:

ID	LAST_NAME	FIRST_NAME	USERID	SALARY
4	Newman	Chad	cnewman	1000
3	Drexler	Ben	bbiri	1100
1	Patel	Ralph	rpatel	1000

Q15. Commit all pending changes. Control data transaction to the MY_EMPLOYEE table.
(Cannot be implemented in Oracle Apex)

SQL QUERY:

```
--15
COMMIT;
```

Q16. Populate the table with the last row of sample data by modifying the statements in the script that you created in Q6.

SQL QUERY:

```
--16
INSERT INTO MY_EMPLOYEE (id, last_name, first_name, userid, salary)
VALUES (5, 'Ropeburn', 'Audrey',
        LOWER(SUBSTR('Audrey', 1, 1) || SUBSTR('Ropeburn', 1, 7)), 1550);
```

OUTPUT:

```
1 row(s) deleted.
```

Q17. Confirm your addition to the table.

SQL QUERY:

```
SELECT * FROM MY_EMPLOYEE;
```

OUTPUT:

ID	LAST_NAME	FIRST_NAME	USERID	SALARY
4	Newman	Chad	cnewman	1000
4	Ropeburn	Audrey	aropebur	1550
3	Drexler	Ben	bbiri	1100
1	Patel	Ralph	rpatel	1000

Q18. Mark an intermediate point in the processing of the transaction. (Cannot be implemented in Oracle Apex)

SQL QUERY:

```
--18
SAVEPOINT point_1;
```

Q19. Empty the entire table.

SQL QUERY:

```
--19
DELETE FROM MY_EMPLOYEE;
```

OUTPUT:

```
4 row(s) deleted.
```

Q20. Confirm that the table is empty.

SQL QUERY:

```
--20
SELECT * FROM MY_EMPLOYEE;
```

OUTPUT:

Results | Explain | Describe | Sa

no data found

Q21. Discard the most recent DELETE operation without discarding the earlier INSERT operation.

(Cannot be implemented in Oracle Apex)

SQL QUERY:

```
--21
ROLLBACK TO point_1;
```

Q22. Confirm that the new row is still intact.
(Cannot be implemented in Oracle Apex)

SQL QUERY:

```
SELECT * FROM MY_EMPLOYEE;
```

Q23. Make the data addition permanent.
(Cannot be implemented in Oracle Apex)

SQL QUERY:

```
--15
COMMIT;
```